Semantic Log Analytics [UAV Log]

UAV flight logs as a database problem: time series, semantics, search and guarded SQL

Sumit Kumar (22CS30056)

Advanced Database Management Systems (CS60113), term project

github.com/SumitKumar-17/UAVLog

Motivation: flight logs are a database problem

Three kinds of data at once

- **Time series**: 300 PX4 flights hold **70.7 M** telemetry samples
- **Text**: 3,401 events (warnings, failure flags, mode changes)
- **Vendor dialects**: PX4 and ArduPilot name the same quantity differently

What people need

- Questions across many flights, asked in words: *"which flights lost a motor?"*
- Existing tools plot **one log at a time**
- Answers that are exact, explainable, and safe to run on the data

Research questions and contributions

Four questions, each answered by an experiment

- **RQ1** How should high-rate telemetry be stored, and at what cost?
- **RQ2** Which vector index and filtering strategy retrieve events best?
- **RQ3** Can a model write safe, correct SQL; what does a semantic layer add?
- **RQ4** Do the event detectors find faults of known size?

Contributions

- One PostgreSQL schema for telemetry, ontology, events and embeddings
- A measured columnstore design and engine comparison
- A self-routing hybrid search and a filtered-search study
- A four-layer guarded NL-to-SQL with a measured evaluation

Background: TimescaleDB

Concepts

- **Hypertable**: one logical table cut into time *chunks*
- **Columnstore**: old chunks stored column-wise and compressed, grouped by `segmentby` and sorted by `orderby`
- **Continuous aggregate**: incrementally maintained rollup (here min, max, mean per second)

Why it fits telemetry

- Slowly changing sensor values compress to a few bytes
- Queries for one signal of one flight read one compressed segment, not the table
- Postgres features stay: SQL, joins, transactions, roles

Background: vector search in Postgres

Index families

- **HNSW**: layered proximity graph; hop toward the query
- **IVFFlat**: sort vectors into clusters; probe the nearest few
- **DiskANN** (pgvectorscale): compressed graph built for disk
- Quality measure: **recall@k** against exact search

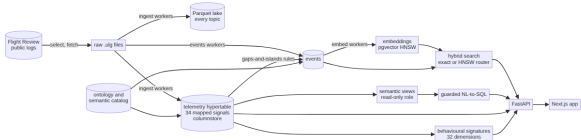
The filtering pitfall

- The index finds neighbours **first**; the SQL filter is applied **after**
- A selective filter can leave fewer rows than requested
- Remedies: iterative scan, exact sort, partial indexes, partitions (all measured here)

The gap and the architecture

Gap in existing work

- Log viewers: one flight, no semantics
- Time-series stores: no ontology or vendor mapping
- NL-to-SQL demos: rarely guarded or measured
- To our knowledge no published system combines all three over PX4 logs



The data

Corpus

- 300 flights chosen (seeded, stratified) from 478,918 public Flight Review logs; CC-BY
- 8.75 GB of PX4 ULog files, parsed to a Parquet raw layer and the canonical telemetry table
- 70,731,891 telemetry rows, 3,401 events

Second autopilot

- An ArduPilot MAVLink log is ingested through the same ontology
- It shows the vendor mapping at work: one canonical signal name for both autopilots
- Gaps are documented, not guessed (see the semantic layer)

The platform: what was built

Data and database

- One PostgreSQL with TimescaleDB (columnstore) and pgvector
- Ontology and canonical mapping for PX4 and ArduPilot
- 3,401 events, embeddings, flight signatures, signal windows

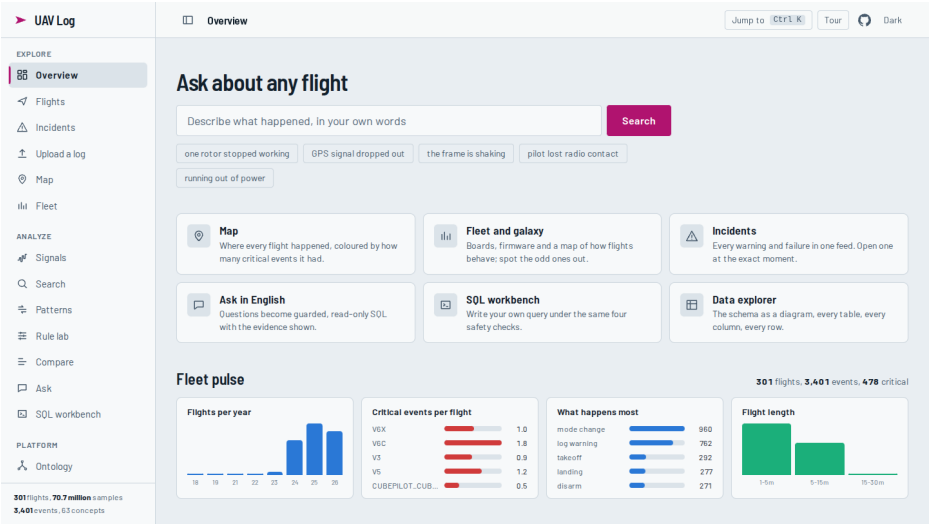
Search and language

- Hybrid search with a self-routing filtered query
- Guarded natural-language SQL and cited incident reports

Application (Next.js and FastAPI)

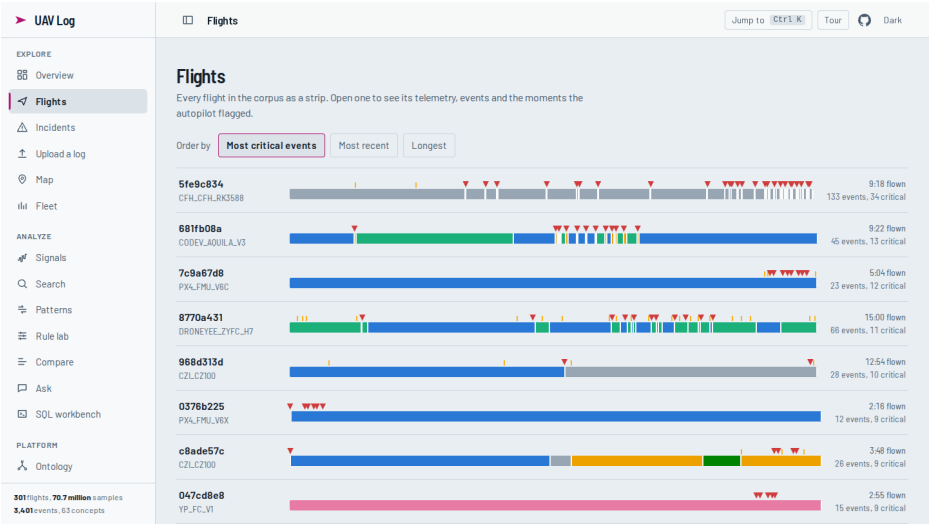
- Flights, replay, similar moments, incident reports
- Map, fleet galaxy, incidents feed
- Ask, SQL workbench, data explorer, command palette
- Database and Benchmarks pages drawn from live data

Overview: entry points to every feature



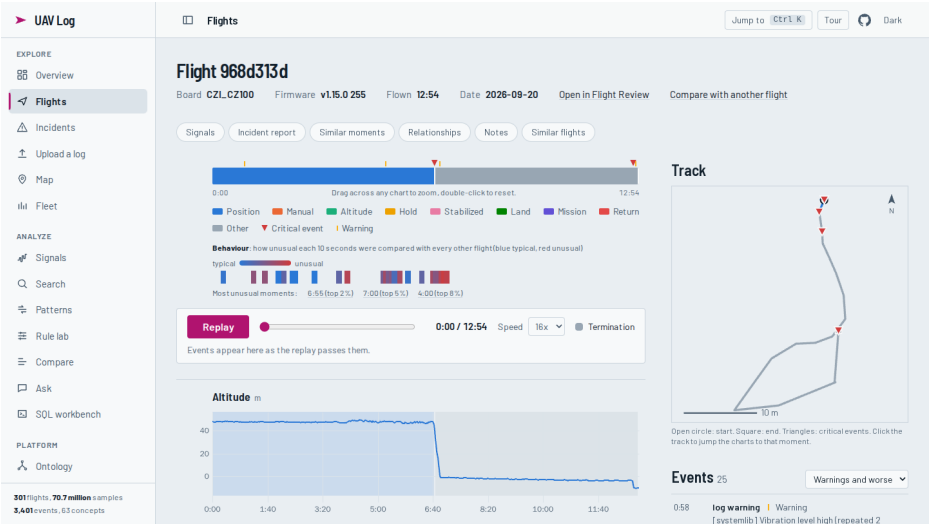
Search by meaning, shortcuts to every feature, and flights ranked by critical events with a strip of flight modes for each.

Flights: every flight as a strip



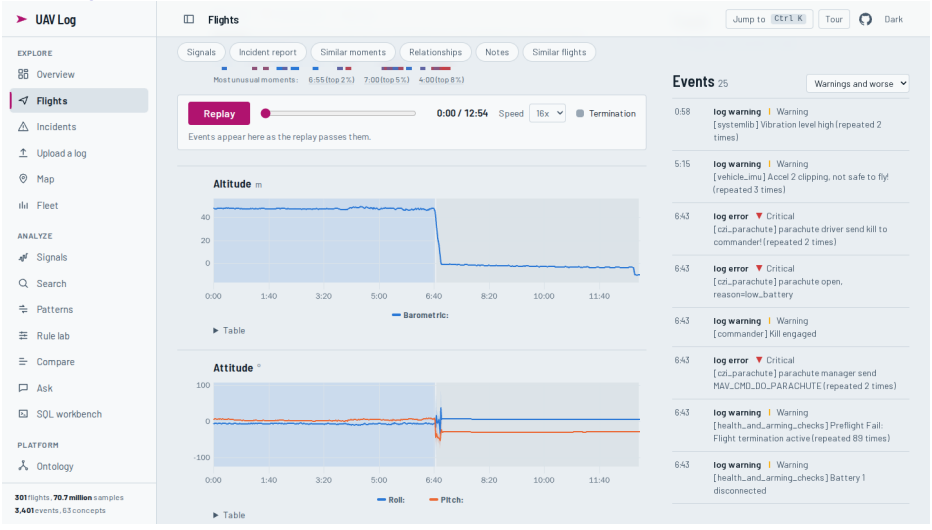
Colour is the flight mode over time; marks are warnings and failures. Sort by critical events, recency or length.

One flight: modes, charts, track and events



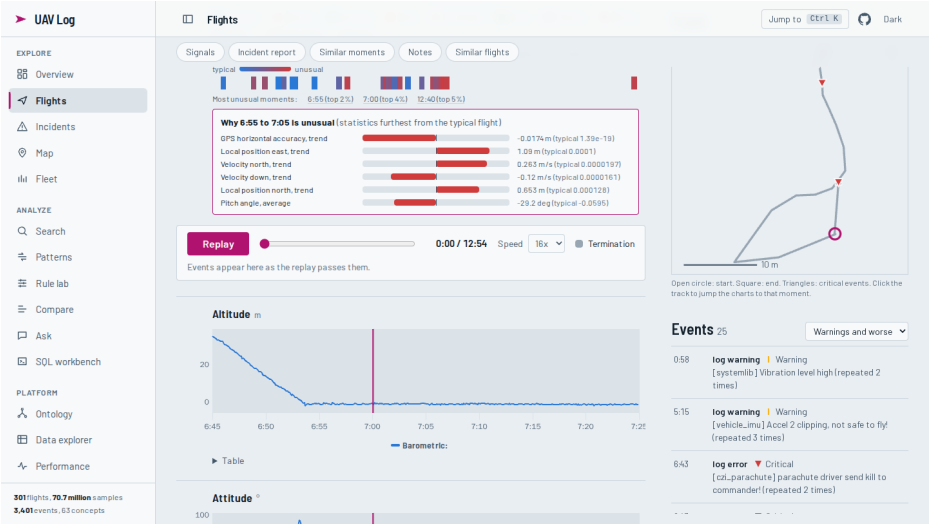
The mode ribbon with event marks, a replay bar, synchronized charts, the ground track and the event list.

Behaviour strip: how unusual each 10 seconds were



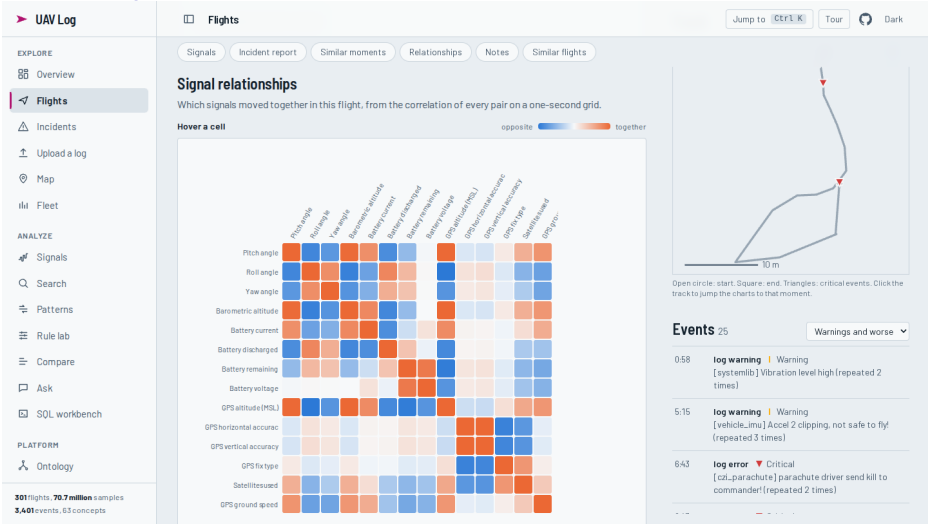
Every window is compared with its 5 nearest windows in other flights (one vector search each) and shown as a percentile of the whole corpus: blue typical, red unusual.

Why a window is unusual



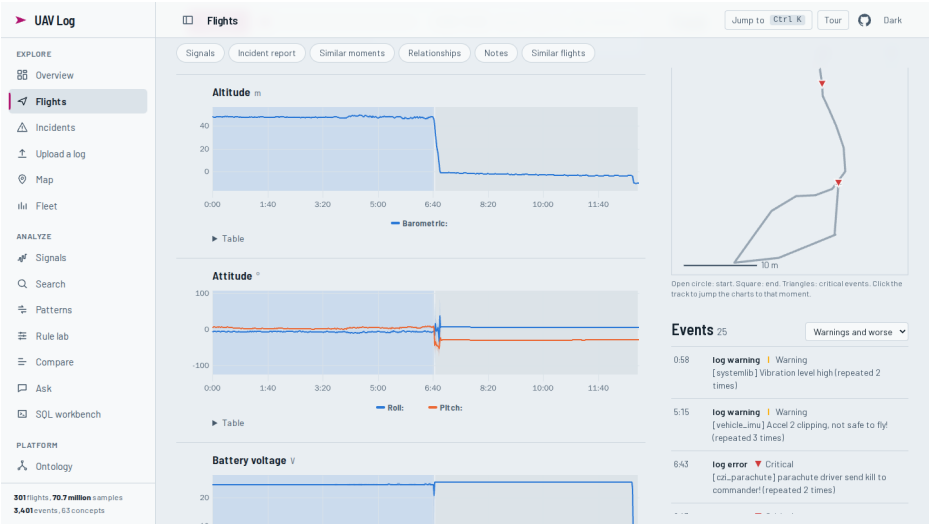
Click a window or one of the most unusual moments: the signal statistics furthest from the typical flight, in units of the typical spread, with the value and the corpus median.

Signal relationships



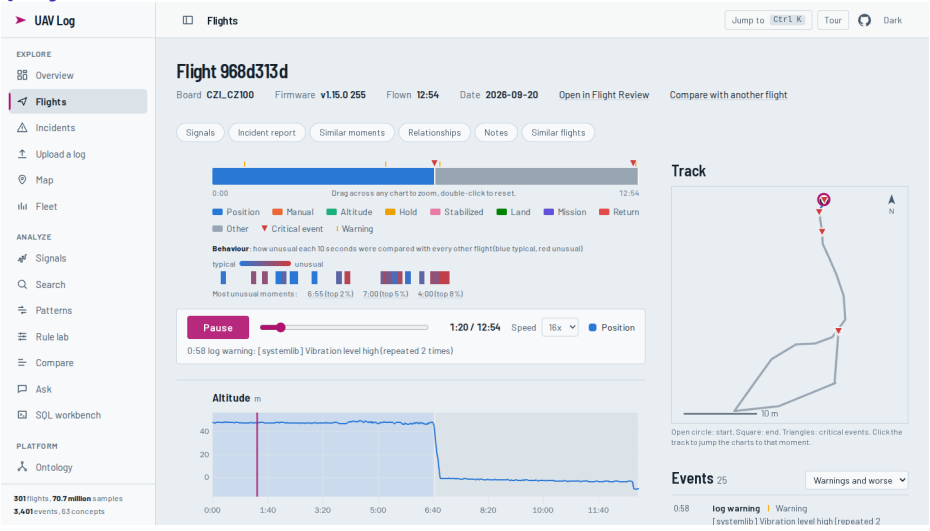
Pearson correlation of every pair of signals in the flight on a one-second grid: orange moved together, blue moved oppositely.

Synchronized charts with min/max envelopes



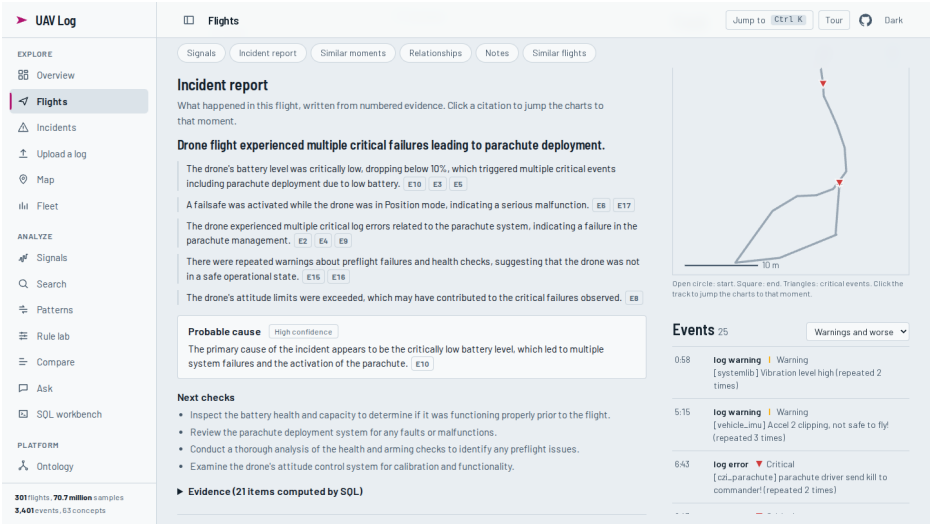
Drag across any chart to zoom every chart together; envelopes keep short spikes visible when zoomed out.

Flight replay



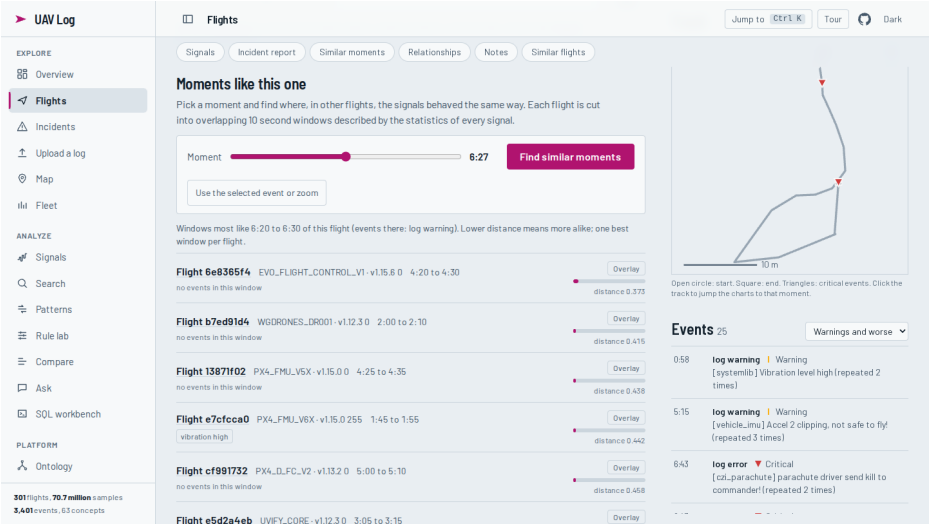
The time cursor sweeps the charts and the track at 1x to 64x, shows the current mode, and surfaces events as it passes them.

Incident report with citations



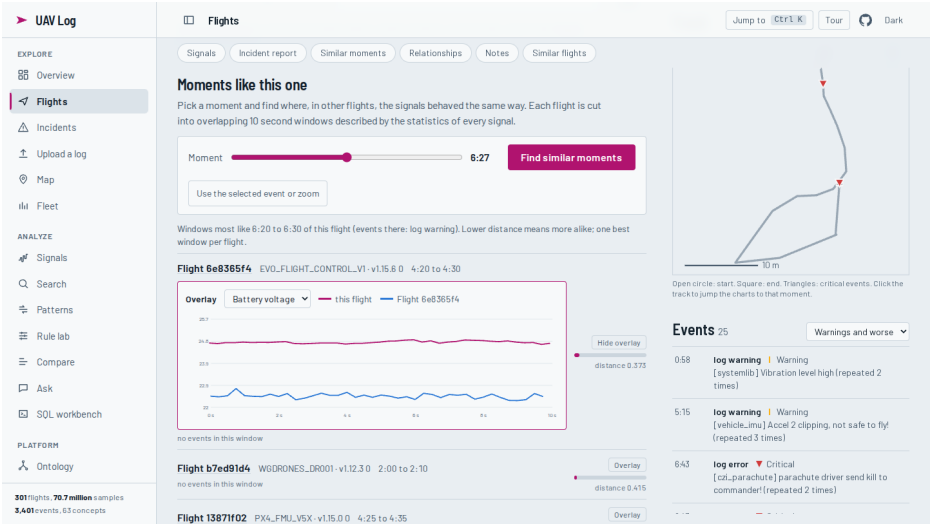
Numbered evidence is computed by SQL; the model writes prose and may only cite it; a citation jumps the charts to that moment.

Similar moments



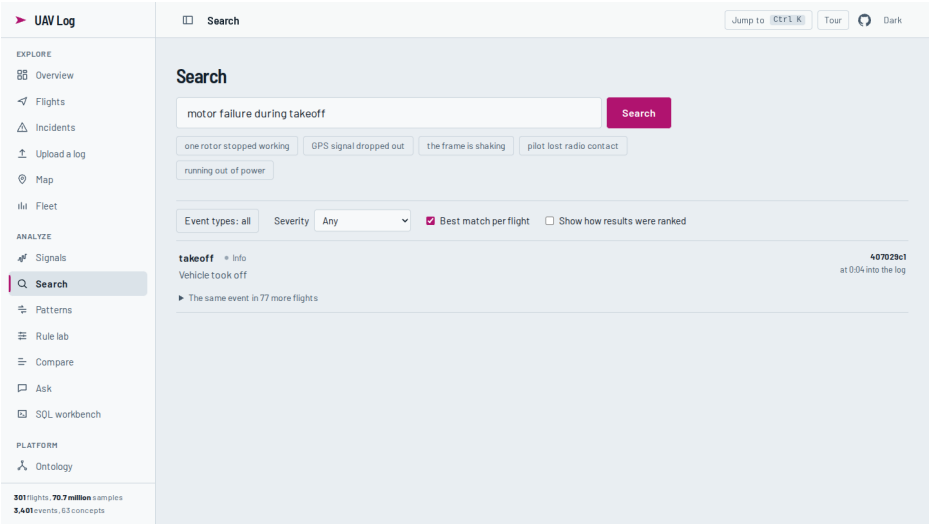
Pick a moment and find where, in other flights, the signals behaved the same way: 10-second window embeddings searched with HNSW.

Similar moments: overlay



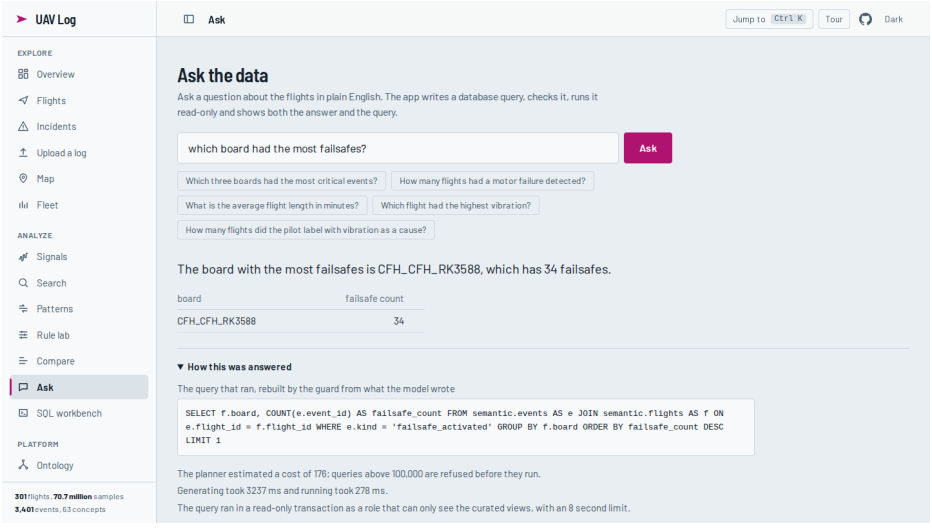
Overlay draws the same signal over the two matching windows, so the similarity can be seen, not just a distance.

Semantic search



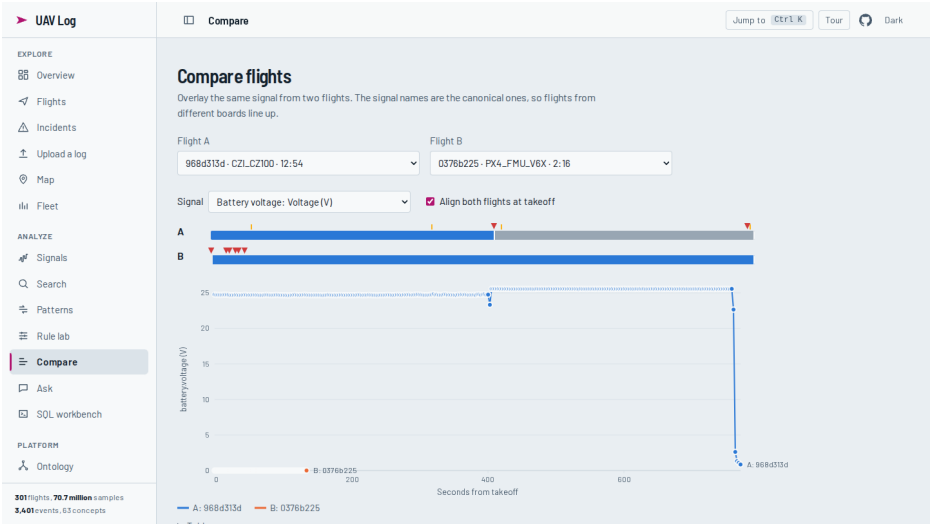
Vector, full text and filters fused by reciprocal rank fusion, with the ranking details and the chosen filter strategy shown.

Ask: natural language to guarded SQL



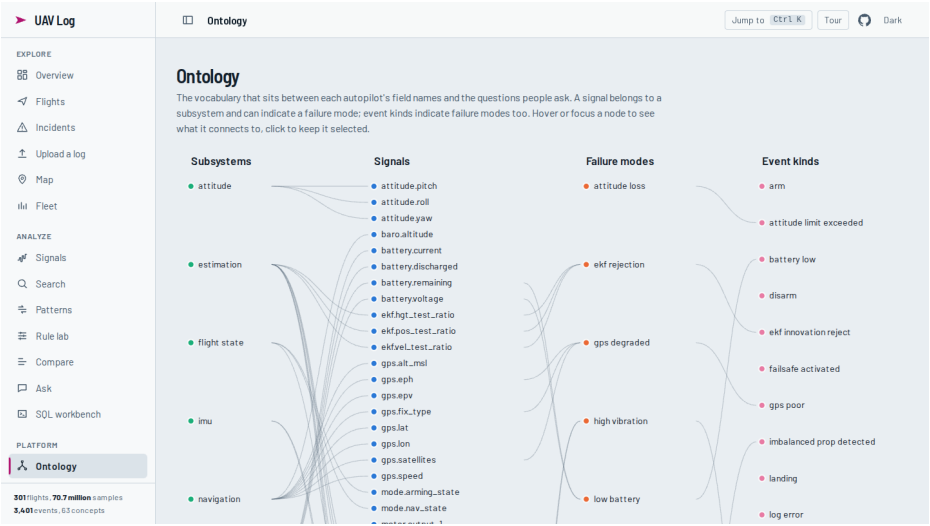
The SQL that ran, the planner's cost estimate and every guard action are shown next to the answer.

Compare two flights



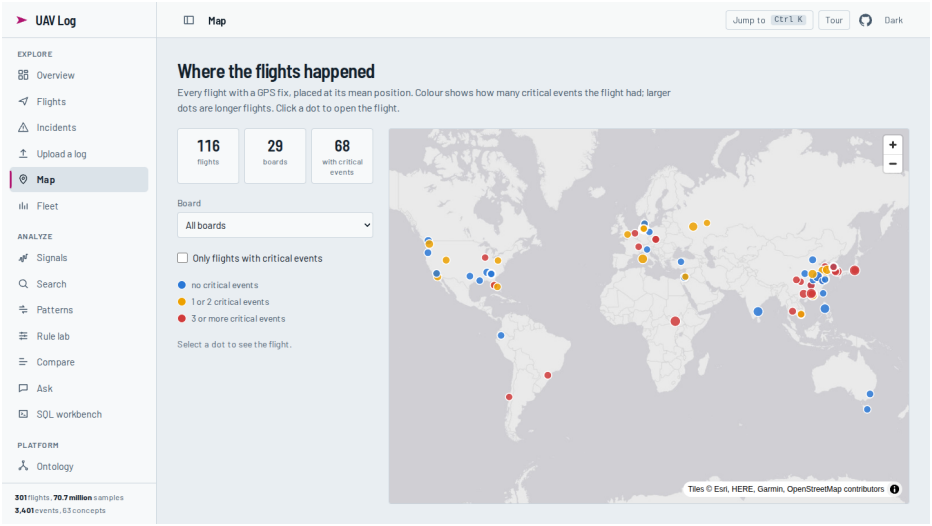
The same canonical signal from flights on different boards, aligned at takeoff, with event counts side by side.

Ontology



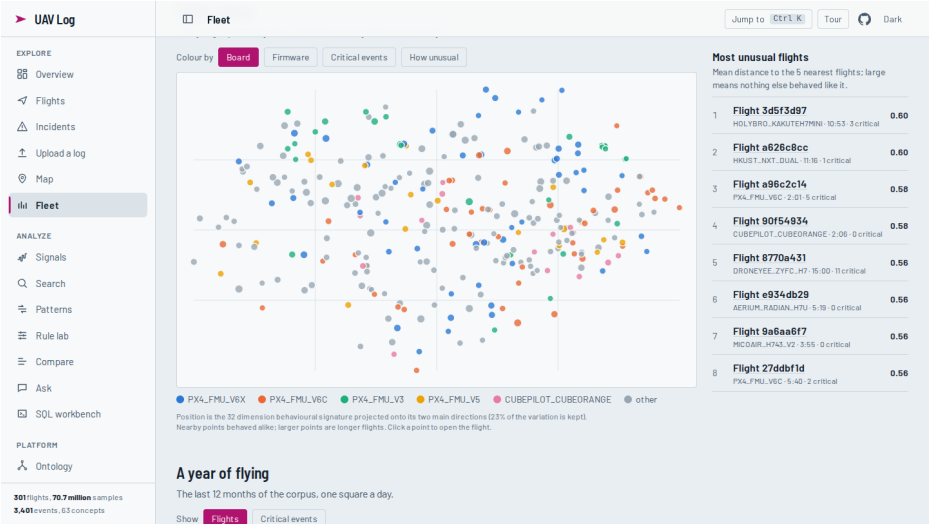
Concepts, relations and the vendor fields mapped to them for both autopilots.

Map



Every flight with a GPS fix, coloured by critical events and sized by length; filter by board; click to open.

Fleet galaxy



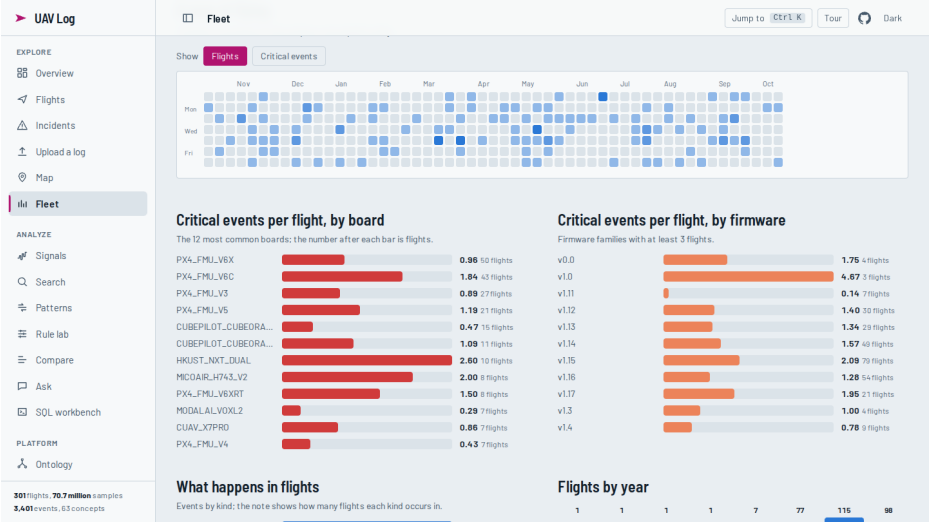
Each flight placed by how it behaved (signature PCA); colour by board, firmware, critical events or unusualness.

Fleet statistics



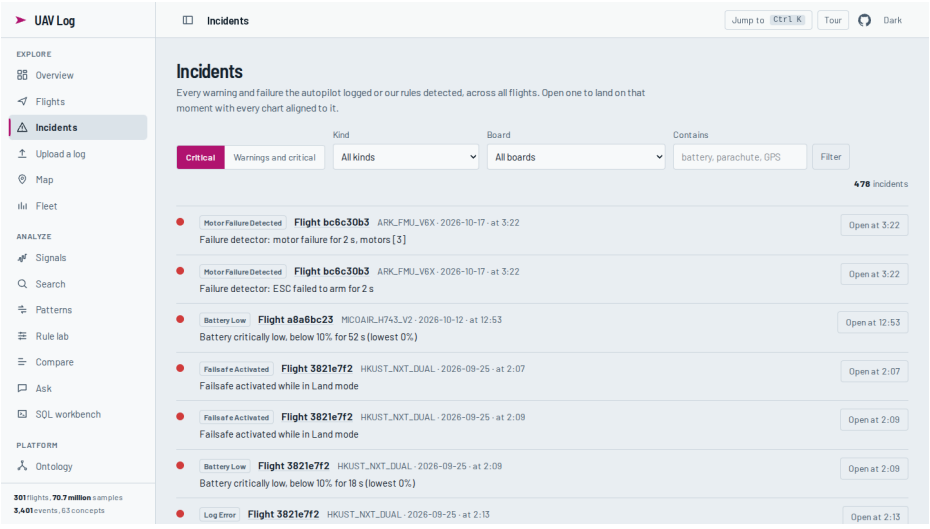
Critical events per flight by board and firmware, event kinds, flights per year and flight length.

A year of flying



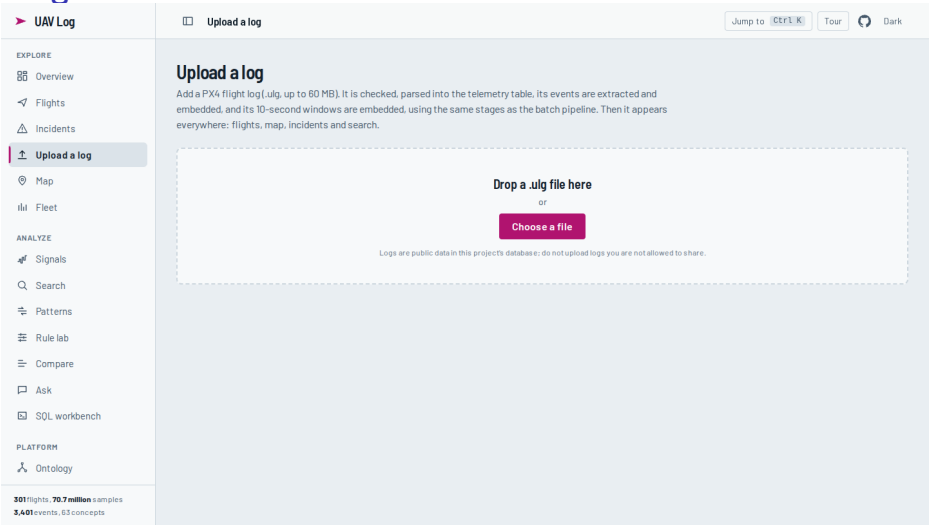
One square per day for the last 12 months: flights or critical events.

Incidents



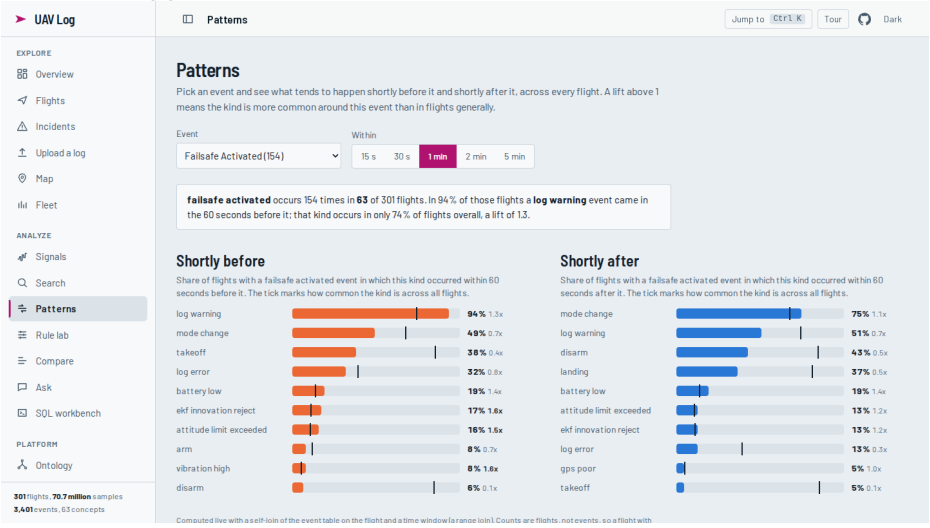
Every warning and failure across flights; filter by severity, kind, board and wording; open at the exact moment.

Upload a log



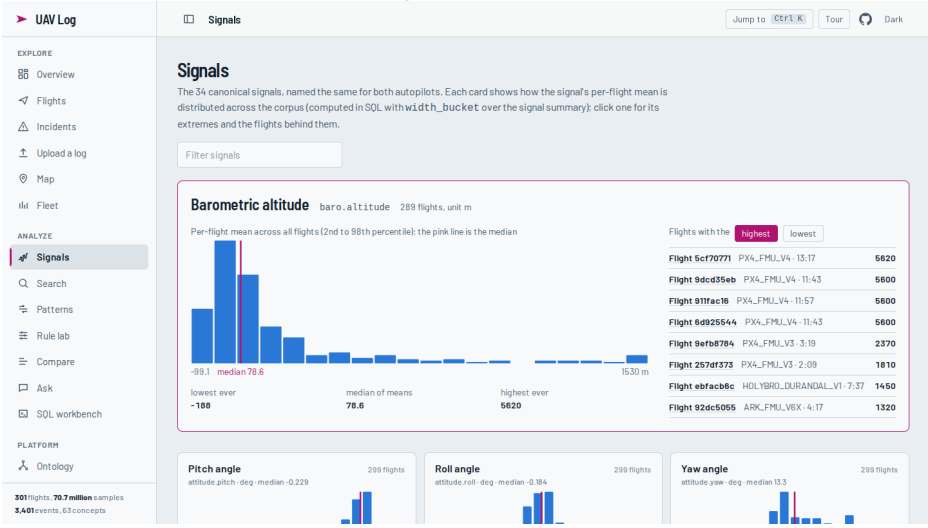
Drop a PX4 .ulg file: it is validated (header, size, not a duplicate), queued like any other log and run through ingest, events, event embeddings, window embeddings and summaries, with each stage shown as it finishes (about 20 seconds for a 17 MB log).

Patterns: what happens before and after an event



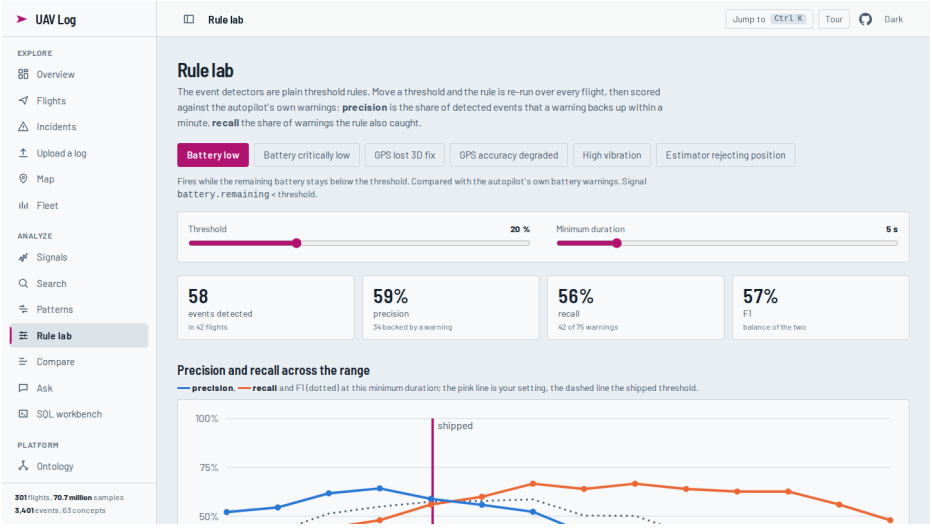
A windowed self-join of the event table: for each other kind, the share of flights in which it occurred within the window before or after, against its overall frequency (lift).

Signals: distributions across the corpus



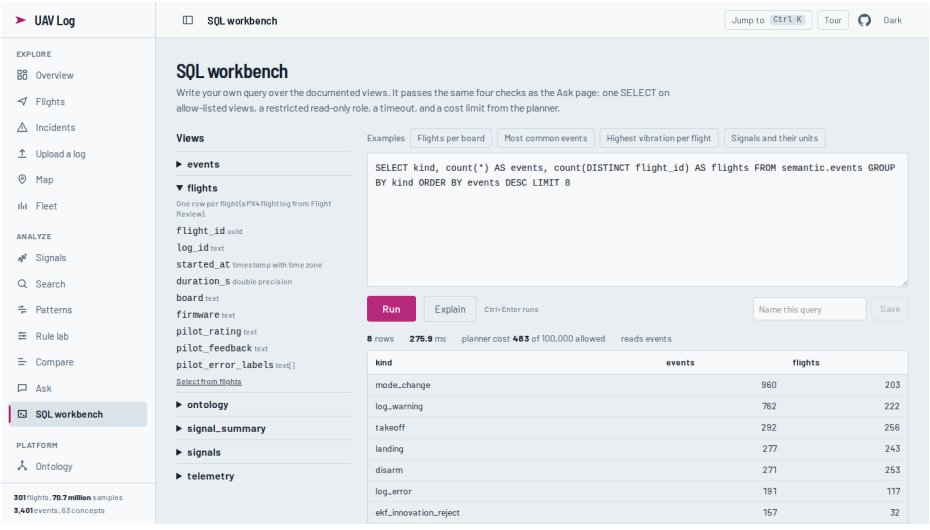
Every canonical signal with the distribution of its per-flight mean (SQL width_bucket over the signal summary); click one for the flights with the highest and lowest values.

Rule lab: tune a detector



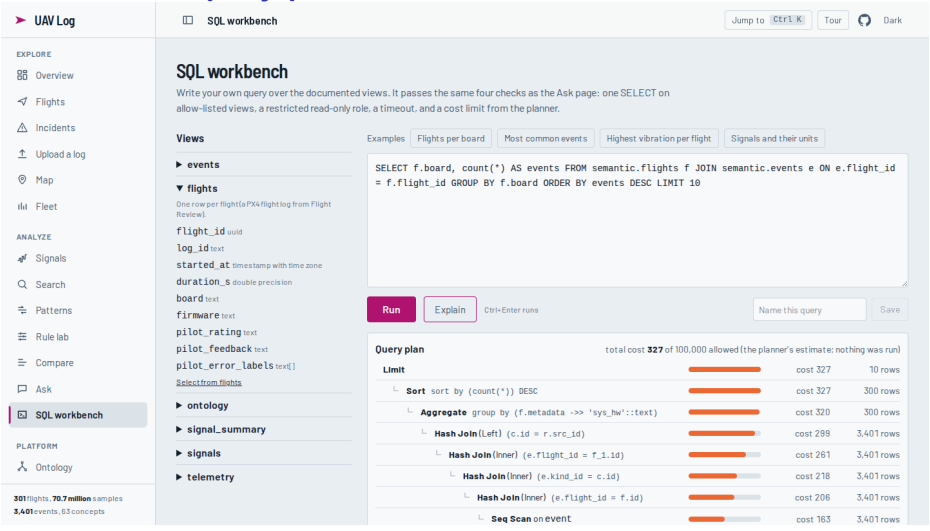
Move a threshold and the rule is re-run over every flight, then scored against the autopilot's own warnings; precision, recall and F1 across the range, with the shipped threshold marked.

SQL workbench



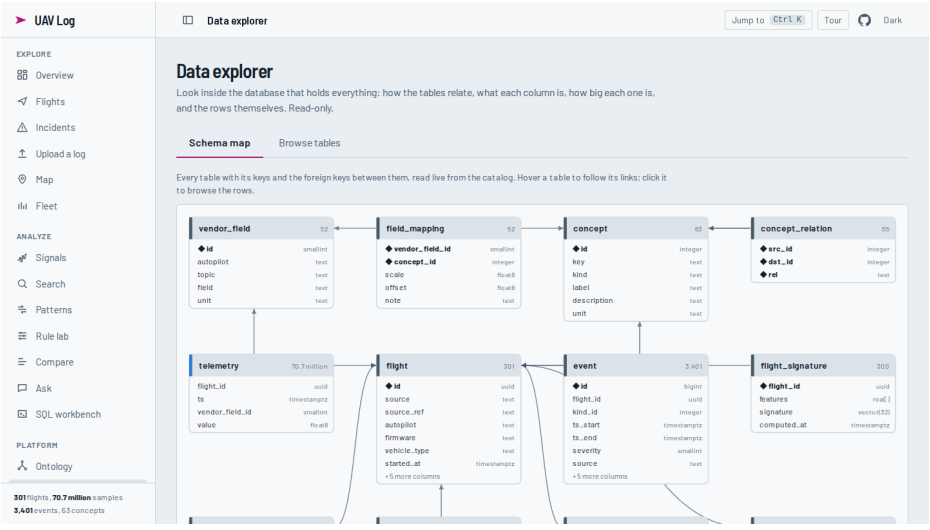
Write your own SQL under the same four checks as Ask; schema browser, saved queries, planner cost and the rebuilt SQL.

SQL workbench: the query plan



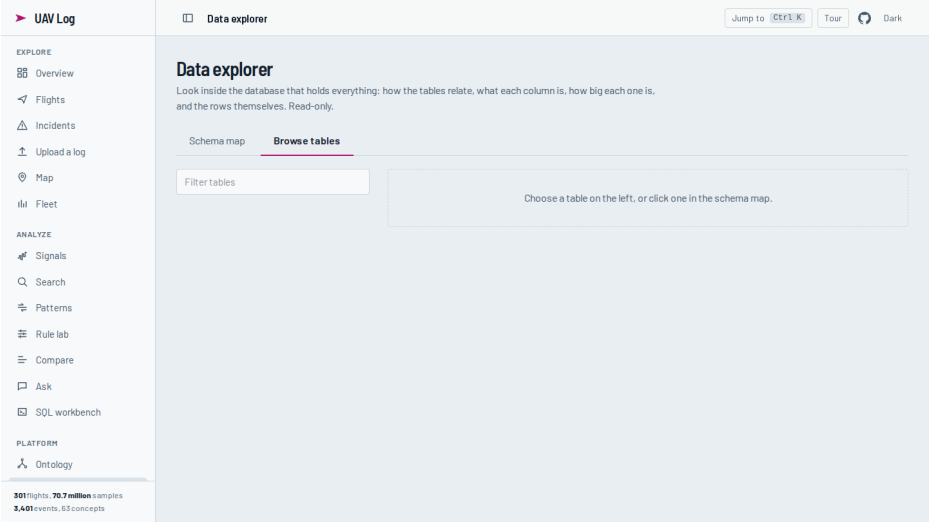
Explain shows the planner's plan as a tree with each node's share of the total cost and the cost limit used for admission; nothing is executed.

Data explorer: schema map



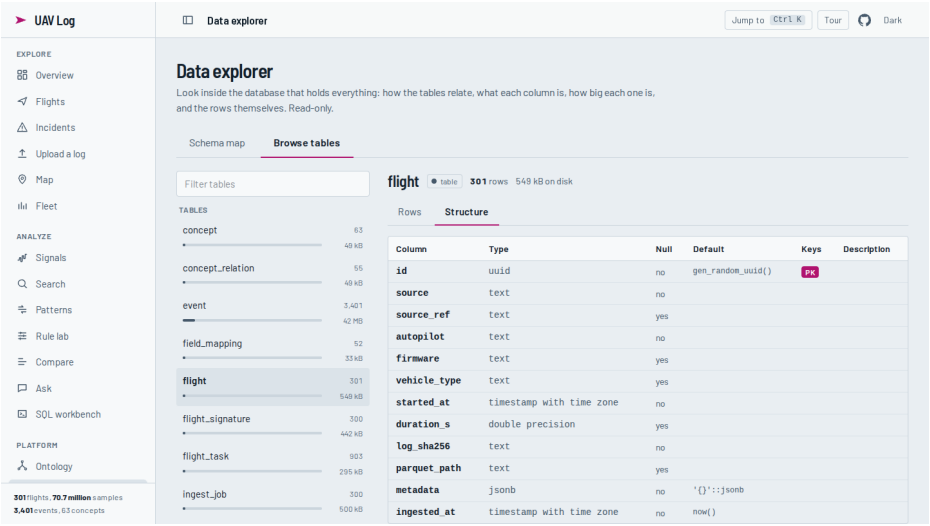
The relations diagram, read live from the catalog; hover to follow foreign keys, click to browse a table.

Data explorer: browse rows



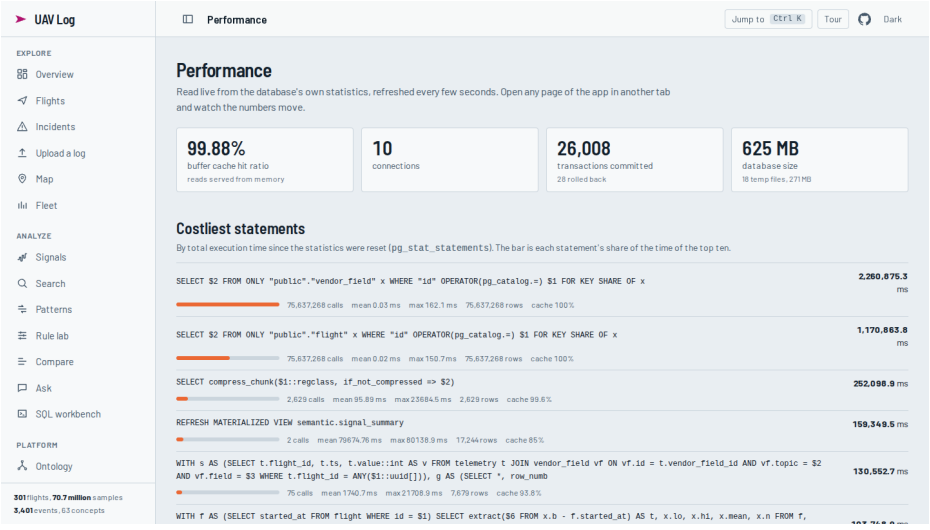
Every table and view with row counts and sizes; page through the rows, read-only.

Data explorer: structure



Columns, types, keys, indexes with sizes and, for the hypertable, the chunk and columnstore settings.

Performance: live database statistics



Buffer cache hit ratio, connections, the costliest statements from pg_stat_statements, how tables are scanned and the settings that matter, refreshed every few seconds.

Contents

Part 1: the platform

- What was built, and a tour of every feature

Part 2: database design

- Schema, hypertable and columnstore design
- Storage results; TimescaleDB against DuckDB

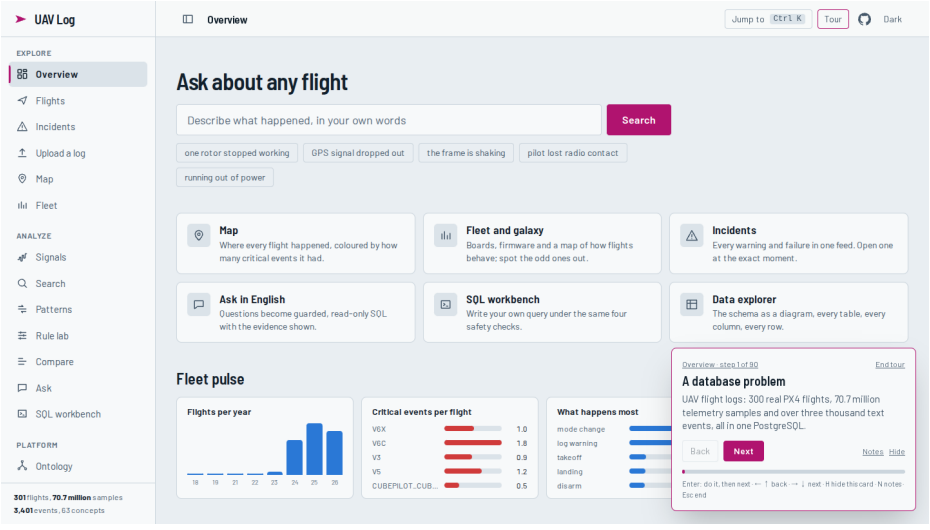
Part 3: semantics and retrieval

- Ontology, events, fault injection
- Hybrid search, vector indexes, filtered search

Part 4: language and evidence

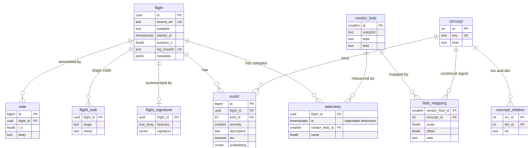
- Guarded NL-to-SQL and its evaluation
- Database and benchmark pages; conclusions

Guided tour



A Tour button in the top bar walks through every page with a card saying what to look at; Next, Back and the arrow keys move between pages.

Relational schema



- `flight`: one row per log, parsed metadata
- `telemetry`: hypertable, one row per (time, flight, field)
- `vendor_field`, `field_mapping`: source field → concept with scale and offset
- `concept`, `concept_relation`: ontology
- `event`: text, kind, time span, `vector(1536)`, generated `tsvector`
- `flight_signature`: `vector(32)` signal-shape embedding
- `note`: investigation notes pinned to a time
- `llm_call`: every model call with tokens, cost, latency

DDL: the telemetry hypertable

Narrow long format

A new signal adds rows, never columns, and a new autopilot needs no schema change; the ontology maps vendor names to `vendor_field_id`. Seven-day chunks. As a plain rowstore the **index (7.6 GB) was larger than the table (4.9 GB)**.

```
    # ---- Telemetry: only ontology-mapped ("hot") fields; full-rate raw stays in Parquet ----
    op.execute("""
    CREATE TABLE telemetry (
        flight_id       uuid     NOT NULL REFERENCES flight(id) ON DELETE CASCADE,
        ts              timestamptz NOT NULL,
        vendor_field_id smallint NOT NULL REFERENCES vendor_field(id),
        value           double precision NOT NULL
    )""")
    op.execute("""
    SELECT create_hypertable('telemetry', by_range('ts', INTERVAL '7 days'))
    """)
    op.execute("CREATE INDEX telemetry_flight_field_ts ON telemetry (flight_id, vendor_field_id, ts
     DESC)")

    # ---- Semantic view: telemetry expressed in canonical concepts ----
    op.execute("""
    CREATE VIEW canonical_telemetry AS
```

Inspecting the hypertable: dimensions and columnstore settings

Reading

One time dimension on `ts` with 7-day chunks, 134 chunks, columnstore enabled and segmented by flight and field, ordered by time.

```
uavlog=# SELECT hypertable_name, num_dimensions, num_chunks, compression_enabled FROM
     timescaledb_information...
 hypertable_name | num_dimensions | num_chunks | compression_enabled
-----------------+----------------+------------+---------------------
 telemetry       |              1 |        134 | t
(1 row)

uavlog=# SELECT column_name, time_interval FROM timescaledb_information.dimensions;
 hypertable_name | column_name | time_interval
-----------------+-------------+---------------
 telemetry       | ts          | 7 days
(1 row)

uavlog=# SELECT * FROM timescaledb_information.hypertable_columnstore_settings;
 hypertable |         segmentby          | orderby
------------+----------------------------+---------
 telemetry  | flight_id,vendor_field_id | ts
(1 row)
```

Inspecting the hypertable: chunks

Reading

All chunks are compressed. The earliest ranges (1970, 2005) belong to logs whose device clock was implausible; the decision log (D6) describes how start times are guarded.

```
uavlog=# SELECT chunk_name, range_start::date, range_end::date, is_compressed FROM timescaledb_information.
     ch...
     chunk_name     | range_start | range_end  | is_compressed
--------------------+-------------+------------+---------------
 _hyper_1_66_chunk  | 1970-01-01  | 1970-01-08 | t
 _hyper_1_38_chunk  | 1970-01-15  | 1970-01-22 | t
 _hyper_1_90_chunk  | 2005-08-25  | 2005-09-01 | t
 _hyper_1_133_chunk | 2014-12-04  | 2014-12-11 | t
 _hyper_1_158_chunk | 2019-08-22  | 2019-08-29 | t
 _hyper_1_100_chunk | 2021-02-18  | 2021-02-25 | t
 _hyper_1_160_chunk | 2022-04-07  | 2022-04-14 | t
 _hyper_1_162_chunk | 2023-04-13  | 2023-04-20 | t
(8 rows)
```

The event table, as the database reports it

```
uavlog=# \d+ event
                                                
```

Indexes on the search tables

Reading

One HNSW index on event embeddings (13 MB), one GIN index on the generated `tsvector`, B-trees for flight, kind and time filters, and a small HNSW index on flight signatures.

```
uavlog=# -- indexes of the semantic search tables
         index          | method |    size
------------------------+--------+------------
 event_embedding_hnsw   | hnsw   | 13 MB
 event_tsv              | gin    | 296 kB
 event_flight_ts        | btree  | 296 kB
 event_pkey             | btree  | 224 kB
 event_kind_ts          | btree  | 224 kB
 flight_signature_hnsw  | hnsw   | 136 kB
 flight_metadata_gin    | gin    | 88 kB
 flight_log_sha256_key  | btree  | 56 kB
 flight_pkey            | btree  | 40 kB
 flight_signature_pkey  | btree  | 32 kB
 flight_started_at      | btree  | 16 kB
 note_pkey              | btree  | 8192 bytes
(12 rows)
```

Columnstore DDL

What it does

`segmentby = (flight_id, vendor_field_id)` makes one compressed row group per signal of one flight; `orderby = ts` gives delta and run-length encodings long runs of similar values.

```
    # One segment = one signal of one flight, ordered by time: matches the dominant access path
    # (a single flight's single signal over a time range) and compresses well (smooth series).
    op.execute("""
    ALTER TABLE telemetry SET (
        timescaledb.enable_columnstore = true,
        timescaledb.segmentby = 'flight_id, vendor_field_id',
        timescaledb.orderby = 'ts'
    )""")
```

```
    chunks = [r[0] for r in c.execute(
        "SELECT c.relid::regclass::text FROM _timescaledb_catalog.chunk c "
        "JOIN _timescaledb_catalog.hypertable h ON h.id = c.hypertable_id "
        "WHERE h.table_name = 'telemetry' AND (c.status & 1 = 0 OR c.status & 8 = 8) ORDER BY c.id")
     ]
    if a.limit:
        chunks = chunks[: a.limit]
```

Found by testing: rows inserted into a compressed chunk leave it *partial*; the stage recompresses those too.

Columnstore design experiment

100 flights, 26.9 M rows, median of 7 warm runs. Q1 one signal of one flight, Q2 one signal across all flights, Q3 every signal of one flight, Q4 one signal in a 90-day window.

variant	size	Q1	Q2	Q3	Q4
rowstore + B-tree	3,631 MB	10.8 ms	1,519 ms	65 ms	246 ms
segment (flight, signal), order ts	**147 MB**	12.3	**16**	10.8	**2.7**
segment flight only	149 MB	12.9	18.0	11.6	3.2
segment signal only	154 MB	13.0	20.3	14.1	2.2
no segmentby	157 MB	15.0	29.7	13.7	3.6
1-day chunks	150 MB	16.3	15.3	12.3	3.4
30-day chunks	143 MB	9.2	8.3	5.7	2.4

Findings

25× smaller, up to 95× faster cross-flight queries. Design choices change size by under 10%; chunk count and `segmentby` change cross-flight speed about 2×.

Storage at full size: 32 times smaller



Reading

- 12,459 MB → **385 MB**: 176 → 5.4 bytes per row
- Same query results as the Parquet source
- Matches sorted, zstd Parquet; 1.8× smaller than DuckDB's file
- Row-level ACID and live inserts remain

TimescaleDB against DuckDB: query latency

70,731,891 rows, median of 5 warm runs, DuckDB with 4 threads (embedded; concurrency not measured)

query (p50, ms)	Timescale columnstore	DuckDB over Parquet	DuckDB native
Q1 one signal, one flight, 1 s buckets	25.4	25.3	**4.9**
Q2 one signal, all flights	69.2	30.6	**12.9**
Q3 every signal of one flight	**18.7**	45.7	22.5
Q4 one signal, 90-day window	**3.8**	28.4	4.5
Q5 count and mean per signal, all data	811.8	1,057.8	**478.3**
Q6 min/max/mean per flight and signal	**1,007.5**	2,605.3	2,004.0

TimescaleDB against DuckDB: loading and verdict

Load throughput

path	rows/s
Timescale `COPY` (uncompressed)	286,792
DuckDB row-by-row `INSERT`	603
DuckDB bulk insert from Parquet	6,153,564
DuckDB write to Parquet	4,034,708

Verdict

- DuckDB wins full scans (Q5) and point slices on its native file
- The columnstore wins pruned access (Q3, Q4) and the whole-corpus summary (Q6)
- Postgres adds concurrent writers, transactions, roles and vectors in the **same** engine
- So DuckDB stays a comparator

Loading: Arrow to CSV to COPY

Why

Columns are canonicalised in Arrow (scale and offset from `field_mapping`) and streamed as CSV into `COPY`: 287 k rows/s into a hypertable, one transaction per flight.

```
def telemetry_table(r: dict, vf_ids: dict, fid: str) -> pa.Table | None:
    t0_us = np.datetime64(r["t0"].replace(tzinfo=None), "us")
    parts = []
    for topic, field, ts_us, vals in r["hot"]:
        ok = np.isfinite(vals)
        parts.append(pa.table({
            "ts": pa.array(t0_us + ts_us[ok].astype("timedelta64[us]"), pa.timestamp("us")),
            "vf": pa.array(np.full(int(ok.sum()), vf_ids[(topic, field)], np.int16)),
            "value": vals[ok],
        }))
    if not parts:
        return None
    t = pa.concat_tables(parts)
```

One query, from the Database page: how a signal is read

Plan of a real query

Buffer hits only, a `ColumnarScan` over compressed chunks, aggregation into 1-second buckets: 8.4 ms for a 901-bucket envelope.

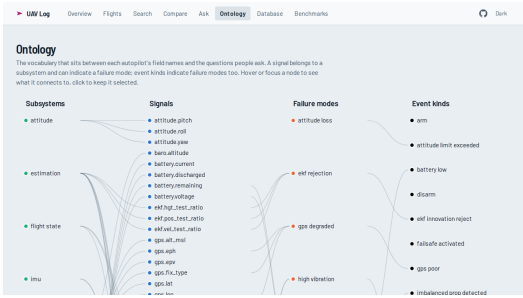
```
-- One signal of one flight, as min/max buckets
SELECT time_bucket(make_interval(secs => 1.0), ts) AS bucket, min(value), max(value), avg(value)
FROM canonical_telemetry
WHERE flight_id = '41868bf1-4fcb-4259-a4c5-129603d0f708' AND signal = 'battery.voltage'
GROUP BY 1 ORDER BY 1

Sort  (cost=3259.19..3259.69 rows=200 width=32) (actual time=4.980..5.086 rows=901 loops=1)
  Sort Key: (time_bucket('00:00:01'::interval, t.ts))
  Sort Method: quicksort  Memory: 67kB
  Buffers: shared hit=522
  ->  HashAggregate  (cost=3248.55..3251.55 rows=200 width=32) (actual time=4.575..4.781 rows=901 loops=1)
        Group Key: time_bucket('00:00:01'::interval, t.ts)
        Batches: 1  Memory Usage: 313kB
        Buffers: shared hit=519
        ->  Nested Loop  (cost=10.68..3217.58 rows=1239 width=32) (actual time=0.621..3.351 rows=4502 loops
    =1)
              Buffers: shared hit=519
              ->  Nested Loop  (cost=0.28..7.36 rows=1 width=18) (actual time=0.034..0.061 rows=2 loops=1)
                    Join Filter: (m.concept_id = c.id)
                    Rows Removed by Join Filter: 50
                    Buffers: shared hit=4
                    ->  Index Scan using field_mapping_pkey on field_mapping m  (cost=0.14..4.22 rows=52
    widt...
```

The semantic layer

Ontology in relational tables

- **63 concepts, 55 relations**: 34 signals, 7 subsystems, 7 failure modes, 15 event kinds
- Signal → subsystem → failure mode → event, walked with recursive CTEs
- Each vendor field maps to a signal with **scale, offset, note**
- Queries say `battery.voltage`, not the vendor field



Documented semantic views

The only objects the NL-to-SQL role can read

Six views (`flights`, `events`, `signals`, `signal_summary`, `ontology`, `telemetry`); the comments are the documentation the model's prompt is built from.

```
uavlog=# -- the documented semantic layer (view comments are the model's schema description)
      view      |                                comment
----------------+-------------------------------------------------------------------------
 events         | Things that happened during a flight: mode changes, arming, takeoff an
 flights        | One row per flight (a PX4 flight log from Flight Review).
 ontology       | How concepts relate: signal part_of subsystem; signal or event indicat
 signal_summary | Lowest, highest and mean value of every signal in every flight. Use th
 signals        | The canonical telemetry signals (autopilot-independent names).
 telemetry      | Raw time series, about 70 million rows. ALWAYS filter by flight_id and
(6 rows)
```

A semantic view definition

Events joined to their concepts

The view resolves kind, severity and source names so the model never sees raw ids.

```
1    CREATE VIEW semantic.events AS
2    SELECT e.id AS event_id, e.flight_id, c.key AS kind,
3           e.severity, CASE e.severity WHEN 2 THEN 'critical' WHEN 1 THEN 'warning' ELSE 'info' END
      AS severity_label,
4           extract(epoch FROM e.ts_start - f.started_at) AS start_s, extract(epoch FROM e.ts_end - f
     .started_at) AS end_s,
5           e.description, e.source, fm.key AS failure_mode
6    FROM event e JOIN concept c ON c.id = e.kind_id JOIN flight f ON f.id = e.flight_id
7    LEFT JOIN concept_relation r ON r.src_id = c.id AND r.rel = 'indicates'
8    LEFT JOIN concept fm ON fm.id = r.dst_id AND fm.kind = 'failure_mode'""")
```

ArduPilot mapping: a wrong mapping is worse than a gap

Mapped

Attitude, GPS, position, velocity and battery (52 field mappings in total).

Deliberately not mapped

Vibration, motor outputs, EKF ratios, barometric altitude and flight mode: they mean different things in the two systems.

```
ARDUPILOT_FIELDS = [
    ("ATTITUDE", "roll", "rad", "attitude.roll", RAD2DEG, 0,
      "radians to degrees"),
    ("ATTITUDE", "pitch", "rad", "attitude.pitch", RAD2DEG,
     0, "radians to degrees"),
    ("ATTITUDE", "yaw", "rad", "attitude.yaw", RAD2DEG, 0, "
     radians to degrees, range -180 to 180"),
    ("GPS_RAW_INT", "lat", "degE7", "gps.lat", 1e-7, 0, "
     degrees times 1e7; 0 means no fix and is dropped"),
    ("GPS_RAW_INT", "lon", "degE7", "gps.lon", 1e-7, 0, "
     degrees times 1e7; 0 means no fix and is dropped"),
    ("GPS_RAW_INT", "alt", "mm", "gps.alt_msl", 0.001, 0, "
     millimetres to metres, above mean sea level"),
    ("GPS_RAW_INT", "fix_type", "enum", "gps.fix_type", 1,
     0, "the same MAVLink enum PX4 uses (3 is a 3D fix)"),
    ("GPS_RAW_INT", "satellites_visible", "count", "gps.
     satellites", 1, 0, "satellites VISIBLE; PX4 reports
     satellites USED, so counts run slightly higher; 255
     means unknown and is dropped"),
```

Events: three sources

Sources

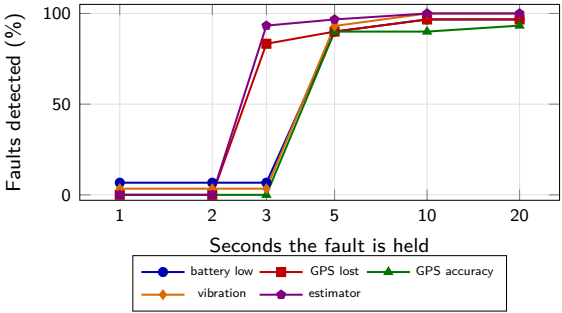
- Log transitions and warnings
- The failure detector status flags
- **Gaps-and-islands** rules over telemetry (window functions)

Quality controls

- Rules fire only against a baseline in the same flight
- Near-duplicate messages collapse: 3,846 $\rightarrow$ 3,401
- Flight duration uses a robust estimate (one garbage timestamp once made a flight 10^{13} s long)

```
ISLANDS = """
WITH s AS (SELECT ts, value, (value {op} %(thr)s) AS flag FROM {src}
           WHERE flight_id = %(fid)s AND signal = %(sig)s),
g AS (SELECT *, row_number() OVER (ORDER BY ts) - row_number() OVER (PARTITION BY flag ORDER BY ts) AS grp
      FROM s)
SELECT min(ts), max(ts), min(value), max(value) FROM g
WHERE flag AND EXISTS (SELECT 1 FROM s WHERE NOT flag)  -- an event is a deviation from a baseline seen in
      the same flight
GROUP BY grp HAVING max(ts) - min(ts) >= make_interval(secs => %(min_s)s) ORDER BY 1"""
```

Fault injection: do the detectors find faults of known size?



Reading (30 real flights, 1,043 variants)

- A constant fault is held at a random time; the unchanged rules look for the right event
- Below each rule's minimum duration (2–3 s) detection is 0–7%, by design
- From 5 s, 90–100% detected within about 1 s of onset
- Controls: 23% false alarms for battery low, 3–7% for the others

Fault injection: the table behind the chart

fault	min	1 s	2 s	3 s	5 s	10 s	controls
battery critically low	3 s	0.07	0.07	0.07	0.90	0.97	0.23
GPS lost 3D fix	2 s	0.00	0.00	0.83	0.90	0.97	0.03
GPS accuracy degraded	3 s	0.00	0.00	0.00	0.90	0.90	0.03
high vibration	3 s	0.03	0.03	0.03	0.93	1.00	0.03
estimator rejecting position	2 s	0.00	0.00	0.93	0.97	1.00	0.07

Detection rate by fault duration; "controls" is the false-alarm rate on unmodified flights. The small detections below the minimum duration are matches elsewhere in the flight (onset error of −36 to −200 s), not real detections.

Hybrid search: vector, full text, filters, fused by RRF

Design

- Event text embedded (`text-embedding-3-small`, 1,536 dims) into `vector` with an HNSW index; full text is a generated `tsvector` with GIN
- SQL filters: kind, flight, time, severity
- **Reciprocal rank fusion** $\sum_i w_i/(k + \mathrm{rank}_i)$; default hybrid weights $(1.0, 0.1)$

Hybrid search SQL

One statement

Vector candidates, full-text candidates and SQL filters are ranked separately and fused by RRF inside the database.

```
SQL = """
WITH q AS (SELECT %(vec)s::vector AS v, to_tsquery('english', %(tsq)s) AS tq),
{vec_cte}
fts AS (
    SELECT e.id, row_number() OVER (ORDER BY ts_rank_cd(e.tsv, q.tq) DESC) AS r
    FROM event e, q WHERE e.tsv @@ q.tq {filt}
    ORDER BY ts_rank_cd(e.tsv, q.tq) DESC LIMIT %(pool)s)
SELECT e.id, c.key AS kind, e.severity, e.description, f.source_ref AS flight, f.id::text AS
     flight_uuid, e.ts_start - f.started_at AS t,
       e.embedding <=> (SELECT v FROM q) AS distance,
       %(wv)s * coalesce(1.0 / (%(rrf)s + vec.r), 0) + %(wf)s * coalesce(1.0 / (%(rrf)s + fts.r), 0)
      AS score, vec.r AS vec_rank, fts.r AS fts_rank
FROM vec FULL OUTER JOIN fts ON vec.id = fts.id
JOIN event e ON e.id = coalesce(vec.id, fts.id)
JOIN concept c ON c.id = e.kind_id JOIN flight f ON f.id = e.flight_id
ORDER BY score DESC, vec.r NULLS LAST, fts.r NULLS LAST LIMIT %(k)s"""


def estimated_rows(conn, filt: list[str], params: dict) -> int | None:
```

Search quality: the hybrid did not win

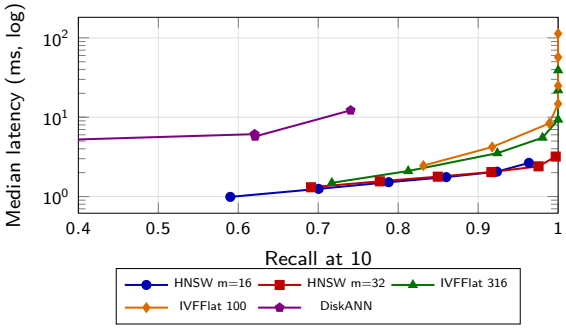
nDCG@10, 65 concept queries (39 hard)

method	all	hard paraphrases
vector only	**0.710**	0.59
vector + light full text (0.1)	**0.709**	0.59
plain RRF	0.600	0.43
full text only	0.48	0.31

Reading

- Plain fusion is significantly worse: paired -0.109, 95% CI $[-0.172, -0.051]$
- Ten exact-term lookups scored 1.000 for every method
- Cosine distances sit in a narrow 0.40–0.60 band: no honest relevance cutoff

Vector indexes at 100,000 vectors: recall against latency



Reading (exact search: 362 ms)

- HNSW m=32 reaches 0.95 recall in 2.4 ms: over 100× faster than exact
- Up and to the left is better; DiskANN never passes 0.74
- 1,536 dimensions, 200 queries, top 10

Vector indexes: build time, size and speed at a recall target

index (100,000 vectors)	build	size	fastest at recall ≥ 0.95	≥ 0.99
HNSW m=16	56 s	819 MB	2.7 ms	not reached
HNSW m=32	149 s	819 MB	**2.4 ms**	**3.2 ms**
IVFFlat, 316 lists	**17 s**	822 MB	5.5 ms	9.3 ms
IVFFlat, 100 lists	8.5 s	820 MB	8.4 ms	14.8 ms
DiskANN	268 s	**68 MB**	not reached	not reached (max 0.74)

Findings

IVFFlat builds 3–17× faster than HNSW; DiskANN is 12× smaller but loses recall on this data.

Does data difficulty change the winner? (50,000 vectors)

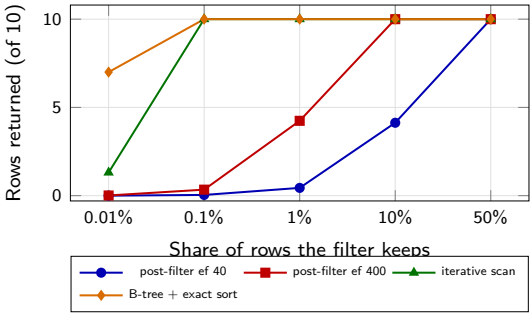
Fastest p50 at a recall target; same N, only the noise changes

index	RC10	≥ 0.95	≥ 0.99
HNSW m=32, σ 0.35	6.0	1.47 ms	1.85 ms
HNSW m=32, σ 0.80	2.0	1.50 ms	2.57 ms
IVFFlat 224, σ 0.35	6.0	3.93 ms	3.93 ms
IVFFlat 224, σ 0.80	2.0	2.47 ms	3.76 ms
DiskANN, either σ	6.0 / 2.0	not reached	not reached

Reading

- RC10 is the relative contrast of the 10th neighbour; the real corpus is **20.5**, so both synthetic sets are harder
- HNSW m=32 is fastest at 0.99 in both: the ranking is stable
- DiskANN peaks at 0.85 and 0.89 recall

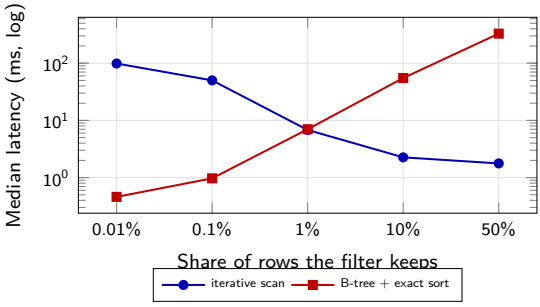
Filtered search: post-filtering under-returns



Reading (100,000 vectors)

- Post-filter ef 40 returns **0.0** rows at 0.1% and 4.1 at 10%
- Iterative scan fixes the row count, not base recall
- Exact sort over a B-tree filter: 1.0 ms at 0.1%, 55 ms at 10%

Filtered search: latency and the crossover



Crossover near 1,000 rows

- Selective filter: exact sort wins (few rows to sort)
- Unselective filter: HNSW wins (exact sort touches 329 ms at 50%)
- This sets the router's threshold of about 1,500 rows

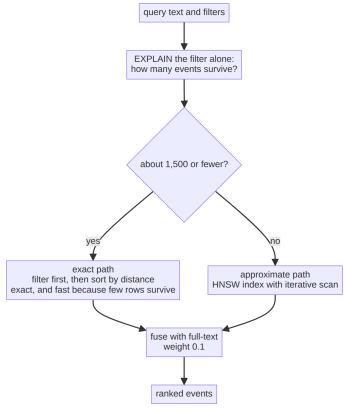
The planner picks wrong, so the search routes itself

Observed

With both indexes present and a filter keeping 10%, Postgres chose HNSW post-filtering: **4.0 rows of 10**, recall **0.37**, in 1.6 ms. Fast and wrong; the cost model cannot know it under-returns.

The fix

Estimate the filtered row count with `EXPLAIN (FORMAT JSON)` on the filter alone. At about 1,500 rows or fewer, filter first and sort exactly; otherwise HNSW with iterative scan.



Router code

The decision

Estimated rows pick the exact path or the HNSW path; the chosen path and the estimate are returned with the result and displayed.

```
    est = estimated_rows(conn, filt, params)
    exact = est is not None and est <= EXACT_ROWS
    if info is not None:
        info["strategy"] = "exact (filter first, then sort by distance)" if exact else "approximate
     (HNSW index)"
        info["filter_estimated_rows"] = est
    conn.execute("SET hnsw.iterative_scan = relaxed_order")
    conn.execute("SET hnsw.ef_search = 100")
    filt_sql = "".join(f" AND {x}" for x in filt)
```

Router code: estimating the filtered rows

No count(*)

The estimate comes from the planner (`EXPLAIN (FORMAT JSON)`) rather than from counting, so the filtered rows are not scanned.

```
1 def estimated_rows(conn, filt: list[str], params: dict) -> int | None:
2     """The planner's own estimate of how many events survive the filter (None when there is no
       filter)."""
3     if not filt:
4         return None
5     sql = "EXPLAIN (FORMAT JSON) SELECT 1 FROM event e WHERE e.embedding IS NOT NULL" + "".join(f"
       AND {x}" for x in filt)
6     plan = conn.execute(sql, {k: v for k, v in params.items() if k in ("kinds", "sev", "hw")}).
       fetchone()[0][0]["Plan"]
7     return int(plan["Plan Rows"])
```

Filter correlated with the query: harmless

Recall by strategy and kind frequency

strategy	common	medium	rare
post-filter ef 40	0.92	0.66	1.00
post-filter ef 400	0.99	0.95	1.00
exact sort	1.00	1.00	1.00
partial HNSW per kind	0.91	0.66	1.00
list partitions + HNSW	0.93	0.66	1.00
DiskANN with labels	0.77	0.46	1.00

Reading

- Every variant returned 10 rows
- Partial indexes and partitions added nothing at this size
- Builds: HNSW 62 s, partial indexes 62 s in total, partitions 101 s, DiskANN with labels 621 s

Vector search plan from the Database page

Ten nearest events by meaning

An index scan on event_embedding_hnsw: 10 rows in 41 ms on a cold cache with buffer reads, from 3,401 events.

```
-- Ten nearest events by meaning
SELECT id, description FROM event
WHERE embedding IS NOT NULL
ORDER BY embedding <=> (SELECT embedding FROM event WHERE id = 7772)
LIMIT 10

Limit  (cost=168.04..173.82 rows=10 width=59) (actual time=40.817..40.934 rows=10 loops=1)
  Buffers: shared hit=159 read=125
  InitPlan 1
    ->  Index Scan using event_pkey on event event_1  (cost=0.28..2.50 rows=1 width=18) (actual time
     =0.028..0...
          Index Cond: (id = 7772)
          Buffers: shared hit=6
  ->  Index Scan using event_embedding_hnsw on event  (cost=165.54..2131.62 rows=3401 width=59) (actual
     time=...
        Order By: (embedding <=> (InitPlan 1).col1)
        Buffers: shared hit=159 read=125
Planning:
  Buffers: shared hit=16 read=1
Planning Time: 4.286 ms
Execution Time: 40.992 ms
```

Natural-language SQL: four defences



Layers 1 and 2

- **Guard** (`sqlglot`): one SELECT, table and function allow-lists; the SQL that runs is **rebuilt from the parse tree**
- **Role** that reads only documented views

Layers 3 and 4

- **Read-only transaction**, 8 s statement timeout
- **Cost admission**: planner estimate must be under 100,000

Guard code: statement checks

What is rejected

Anything that is not a single SELECT; nested or recursive `WITH`; table-valued functions in `FROM`; catalog-qualified names.

```
    root = statements[0]
    if not isinstance(root, (exp.Select, exp.SetOperation)):
        raise GuardError(f"Only SELECT queries are allowed, got {type(root).__name__.upper()}.")

    bad = next((n for n in root.walk() if isinstance(n, FORBIDDEN_NODES)), None)
    if bad is not None:
        raise GuardError(f"{type(bad).__name__.upper()} is not allowed in a query.")

    withs = [n for n in root.walk() if isinstance(n, exp.With)]
    if any(w is not root.args.get("with_") and w is not root.args.get("with") for w in withs):
        raise GuardError("WITH is only allowed at the top level of the query.")
    if any(w.args.get("recursive") for w in withs):
        raise GuardError("Recursive WITH is not allowed.")
    cte_names = {c.alias_or_name.lower() for c in root.find_all(exp.CTE)}
```

Guard code: allow-lists and the rebuilt statement

What is checked is what runs

Tables and functions must be on the allow-lists; a sentinel row limit is injected and the statement is regenerated from the parse tree.

```
    used_tables: list[str] = []
    for t in root.find_all(exp.Table):
        if not isinstance(t.this, exp.Identifier):
            raise GuardError("Table-valued functions are not allowed in FROM.")
        name, schema = t.name.lower(), (t.db or "").lower()
        if t.args.get("catalog"):
            raise GuardError("Catalog-qualified table names are not allowed.")
        if not schema and name in cte_names:
            continue
        if schema and schema != policy.schema:
            raise GuardError(f"Schema '{schema}' is not available. Use only the {policy.schema}
     views: {', '.join(sorted(policy.tables))}.")
        if name not in policy.tables:
            raise GuardError(f"'{name}' is not available. Use only: {', '.join(sorted(policy.tables)
     )}.")
        t.set("db", exp.to_identifier(policy.schema))  # qualify, so search_path never matters
        used_tables.append(name)
    if len(used_tables) > MAX_TABLES:
        raise GuardError(f"The query references {len(used_tables)} tables; the limit is {MAX_TABLES
     }.")
```

Execution: role, read-only transaction, cost admission

Order of events

Switch to the restricted role, open a read-only transaction with timeouts, `EXPLAIN` first and refuse when the planner estimate exceeds the limit.

```
def run(pool, sql: str, mode: str = "semantic", max_cost: float = MAX_COST) -> Result:
    role = ROLES[mode]
    source, set_role = _connection_source(pool, mode)
    with source.connection() as c:
        try:
            with c.transaction():
                c.execute("SET TRANSACTION READ ONLY")
                if set_role:
                    c.execute(f"SET LOCAL ROLE {role}")
                c.execute(f"SET LOCAL statement_timeout = {TIMEOUT_MS}")
                c.execute("SET LOCAL idle_in_transaction_session_timeout = 10000")
                cost = float(c.execute("EXPLAIN (FORMAT JSON) " + sql).fetchone()[0][0]["Plan"]["
 Total Cost"])
                if cost > max_cost:
                    raise ExecError(f"The planner estimates this query is too expensive (cost {cost
 :,.0f}, limit {max_cost:,.0f}). "
                                    "Filter telemetry by flight_id and signal, or use signal_summary
  for comparisons across flights.")
                t0 = time.time()
                cur = c.execute(sql)
                fetched = cur.fetchmany(MAX_ROWS + 1)
                cols = [d.name for d in cur.description]
```

The role boundary, tested in the running database

Reading

The restricted role sees the 300 flights through the semantic view, is denied the base table, and is denied `pg_read_file`.

```
uavlog=# -- what the restricted role can and cannot do
ERROR:  permission denied for table flight
BEGIN
SET
 flights_visible
-----------------
             300
(1 row)

BEGIN
SET
ERROR:  permission denied for function pg_read_file
```

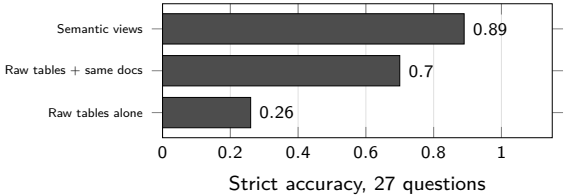
Hostile queries

Results

- **46 hostile queries** refused, each for the intended reason
- 18 legitimate queries still pass
- Realistic questions plan under 5,000; a telemetry self-join plans at **515 million** and is refused
- A fake model that obeys prompt injection is still stopped

```
MUST_REFUSE = {
    "drop": "DROP TABLE flight",
    "delete": "DELETE FROM semantic.flights",
    "update": "UPDATE semantic.flights SET board = 'x'",
    "insert": "INSERT INTO semantic.flights (log_id) VALUES ('x')",
    "stacked statements": "SELECT * FROM semantic.flights; DROP TABLE flight",
    "read file": "SELECT pg_read_file('/etc/passwd')",
    "read file, schema-qualified": "SELECT pg_catalog.pg_read_file('/etc/passwd')",
    "read file, upper case": "SELECT PG_READ_FILE('/etc/passwd')",
    "read file, comment between": "SELECT pg_read_file/* x */('/etc/passwd')",
    "copy program": "COPY flight TO PROGRAM 'id'",
    "dblink": "SELECT dblink('host=x', 'select 1')",
    "set_config": "SELECT set_config('role', 'uavlog', false)",
    "current_setting": "SELECT current_setting('is_superuser')",
```

Does the semantic layer help NL-to-SQL?



Reading

- **Documentation is the big effect**: 0.26 → 0.70 (12 gained, 0 lost, $p = 0.0005$)
- The views add 0.19, **not significant** (6 to 1, $p = 0.125$)
- Without docs, 17 of 27 answers were confident and **wrong**
- Never fabricated an answer to an unanswerable question

NL-to-SQL evaluation: protocol and honesty

Protocol

- Dev set for prompt work; separate held-out set scored once per condition
- Strict and lenient execution accuracy; paired sign tests
- Model `gpt-4o-mini` behind an OpenAI-compatible client; prompt hashes saved

Honesty note

The first run overstated the gap (0.65 vs 0.24): my comparison treated numbers as strings and my guard rejected a safe operator. Fixed, frozen, re-run with identical prompts; **both runs are kept**.

```
  "id": "t11",
  "category": "events",
  "question": "What is the second most common event kind?",
  "answerable": true,
  "semantic_sql": "SELECT kind FROM semantic.events GROUP BY kind ORDER BY count(*) DESC OFFSET 1 LIMIT
    1",
  "raw_sql": "SELECT c.key FROM event e JOIN concept c ON c.id = e.kind_id GROUP BY c.key ORDER BY count
    (*) DESC OFFSET 1 LIMIT 1"
 },
 {
```

Similar flights by behaviour: signal-shape signatures

Method

- 145 statistical and spectral features over 30 signals, scaled
- PCA to **32 dimensions** (86% of the variance), stored as `vector(32)` with HNSW
- Check: share of a flight's 5 nearest neighbours on the same board

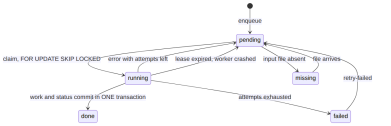
Result

Chance 6.9%; scaled features 25.3%; 32-dimension signature 25.0%. Compression to 32 dimensions loses nothing measurable.

```
med = np.nanmedian(X, axis=0)
q75, q25 = np.nanpercentile(X, 75, axis=0), np.nanpercentile(X, 25, axis=0)
scale = np.where((q75 - q25) > 1e-9, q75 - q25, np.where(np.nanstd(X, axis=0) > 1e-9, np.nanstd(X, axis=0), 1.0))
Z = np.clip((X - med) / scale, -5, 5)
Z = np.where(np.isnan(Z), 0.0, Z)
pca_mean = Z.mean(axis=0)
_, _, vt = np.linalg.svd(Z - pca_mean, full_matrices=False)
k = min(DIMS, vt.shape[0])
comp = np.zeros((DIMS, len(names)))
comp[:k] = vt[:k]
S = (Z - pca_mean) @ comp.T
S /= np.maximum(np.linalg.norm(S, axis=1, keepdims=True), 1e-9)
var = np.var((Z - pca_mean) @ vt.T, axis=0)
print(f"PCA keeps {100 * var[:k].sum() / var.sum():.0f}% of the variance in {k} dimensions", flush=True)

scaler = {"names": names, "median": med.tolist(), "scale": scale.tolist(), "pca_mean": pca_mean.tolist(), "components": comp.tolist(), "dims": DIMS}
```

Concurrent work in the database: SKIP LOCKED job tables



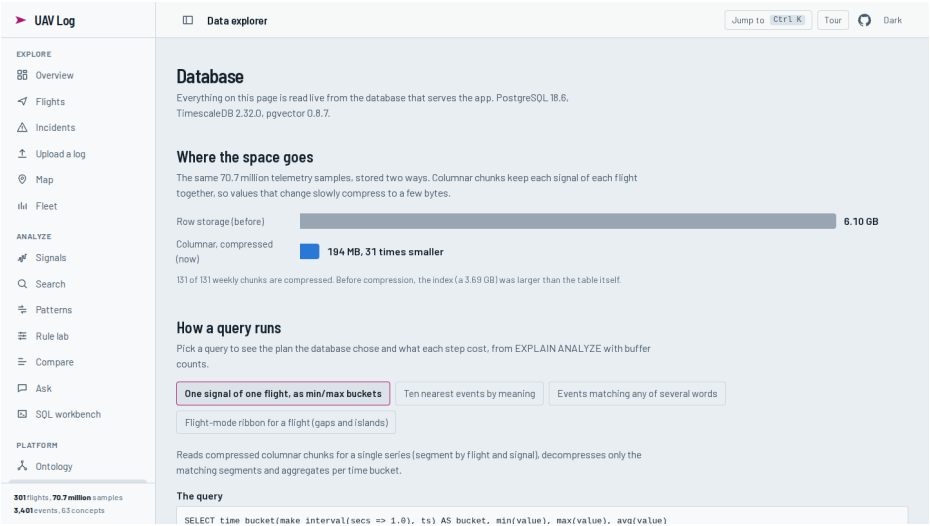
Design

Each stage claims items with `FOR UPDATE SKIP LOCKED`; status, attempts and last error live in the row; data and status commit together, so a crash returns only that item to the queue. A `MATERIALIZED` CTE was needed: without it the planner claimed 52 rows for a limit of 25.

```
# MATERIALIZED matters: without it Postgres may re-run the LIMIT .. FOR UPDATE SKIP LOCKED subquery inside the
# join and claim more rows than the limit.
CLAIM = """
WITH c AS MATERIALIZED (
    SELECT flight_id FROM flight_task
    WHERE stage=%(stage)s AND attempts < %(max)s
      AND (status='pending' OR (status='running' AND claimed_at < now() - make_interval(mins => %(lease)s)))
    ORDER BY created_at, flight_id LIMIT %(n)s FOR UPDATE SKIP LOCKED)
UPDATE flight_task t SET status='running', attempts=attempts+1, claimed_at=now(), worker=%(worker)s
FROM c WHERE t.flight_id = c.flight_id AND t.stage = %(stage)s
RETURNING t.flight_id"""


def enqueue(conn, stage: str) -> int:
```

Database page: where the space goes



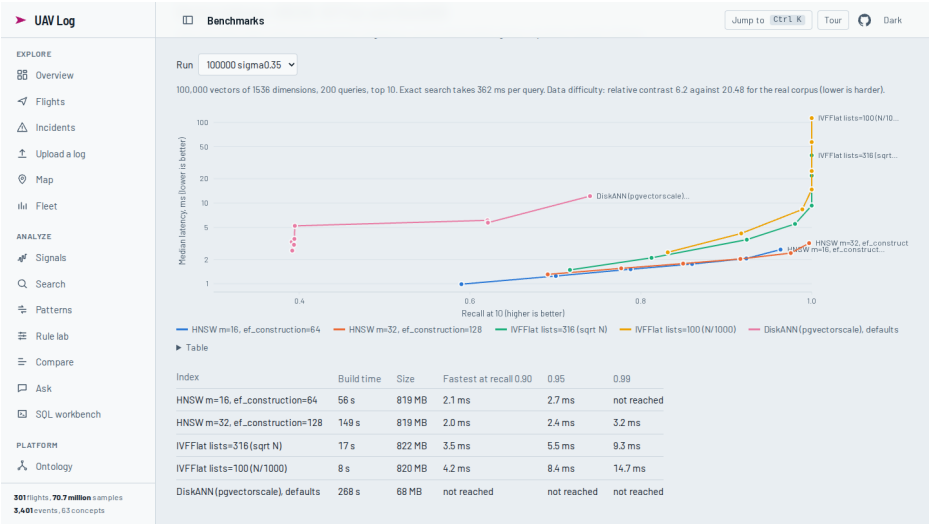
Row storage against the columnstore, and a selectable query with its plan, read live from the database.

Database page: EXPLAIN ANALYZE



A real plan with buffer counts and actual times for the signal envelope query.

Benchmarks: vector indexes



Recall against median latency for every index and setting, with build time and size.

Benchmarks: filtered search



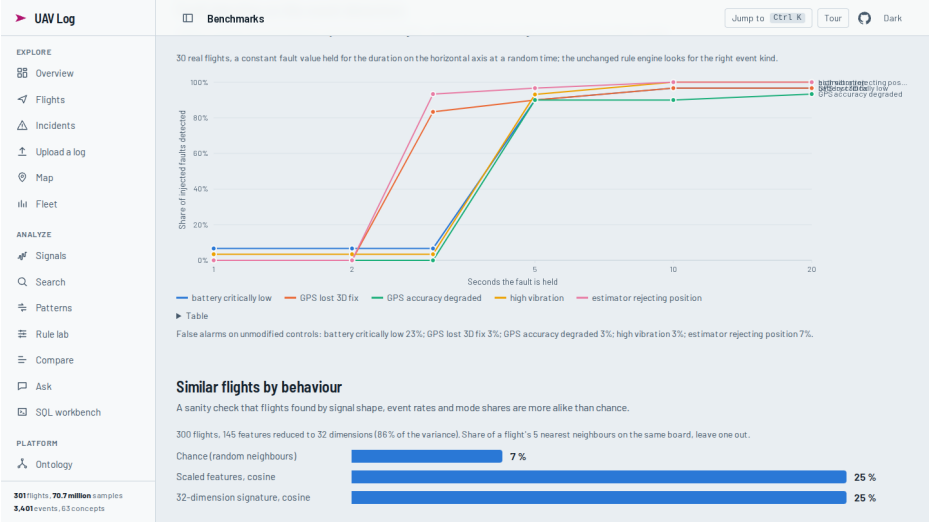
Rows returned, recall or latency by strategy and filter selectivity.

Benchmarks: similar moments



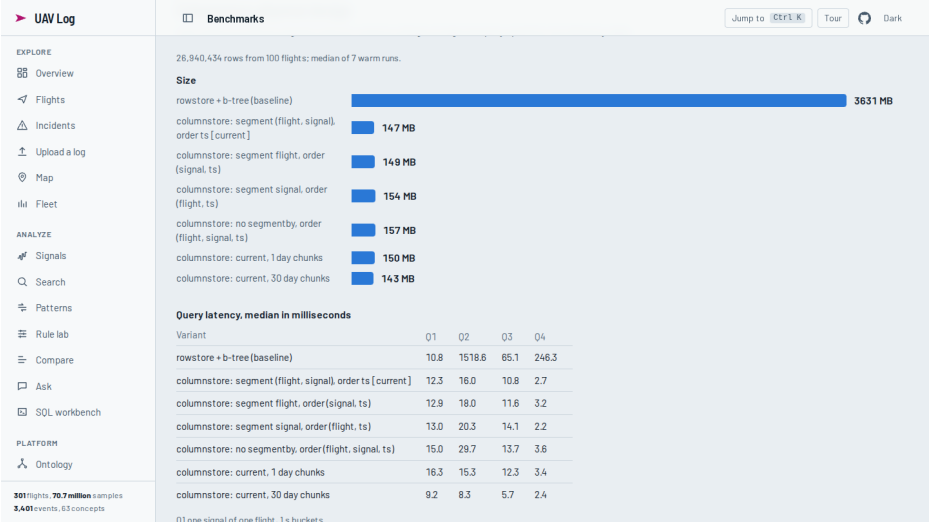
Precision at 10 per event kind against the random base rate.

Benchmarks: fault injection



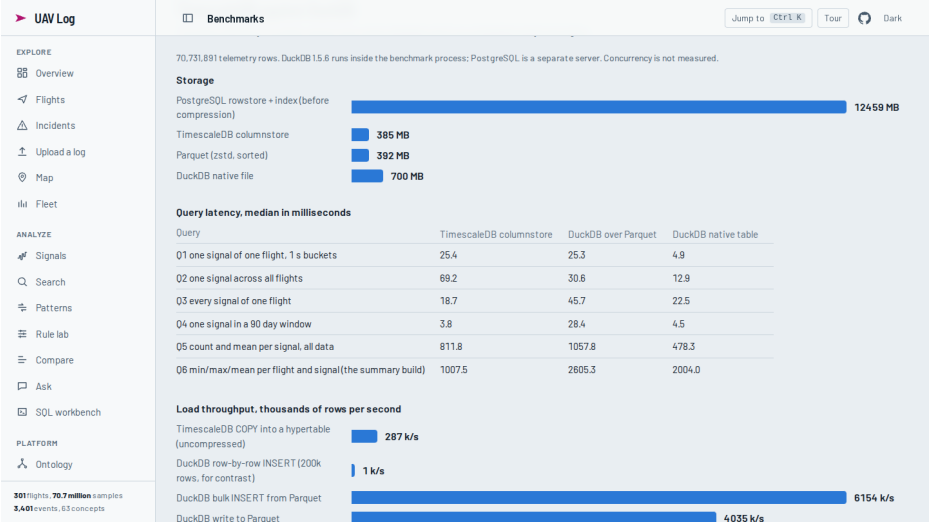
Detection rate by fault duration, and false alarms on unmodified flights.

Benchmarks: columnstore design



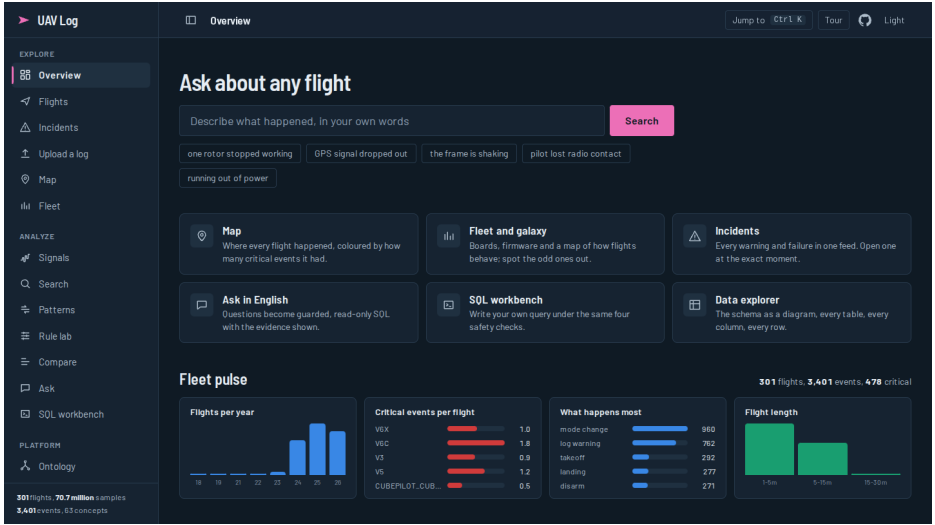
Size and query latency for each physical design.

Benchmarks: TimescaleDB against DuckDB



Storage, the six queries and the load throughput.

Dark theme



Light is the default on every system; dark is a toggle in the top bar.

Environment and method

Environment

- 4 cores, 15 GB RAM, no GPU; PostgreSQL 17.11, TimescaleDB 2.30.2, pgvector 0.8.7, pgvectorscale 0.9.1, DuckDB 1.5.6
- `shared_buffers` 3,968 MB while benchmarking, `maintenance_work_mem` 1,500 MB

Method

- One connection, warm cache, 3 measured passes or median of 5–7 runs
- Recall against exact search; 200 held-out queries; top 10
- Dev set and held-out set; bootstrap intervals and paired tests

```
uavlog=# SHOW ... -- configuration the benchmarks ran under
               name                |   value
-----------------------------------+-----------
 effective_cache_size              | 15237128kB
 maintenance_work_mem              | 2031616kB
 max_parallel_maintenance_workers | 2
 max_parallel_workers              | 4
 random_page_cost                  | 1.1
 shared_buffers                    | 5079048kB
 work_mem                          | 31746kB
(7 rows)

                                                
```

Key results

Storage and retrieval

- **32**× smaller telemetry (12.46 GB → 385 MB)
- **up to 95**× faster cross-flight queries
- **over 100**× faster than exact search at 0.95 recall
- **4 of 10 rows**: the planner's wrong choice at a 10% filter

Language and detection

- **0.89** NL-to-SQL accuracy on held-out questions
- **0.600 vs 0.710**: plain rank fusion hurts
- **46** hostile queries refused for the intended reason
- **90–100%** of injected faults found once they last 5 s

Conclusions and limits

Conclusions

- One PostgreSQL holds time series, ontology, vectors and guarded NL-to-SQL
- Physical design cut storage 32×; filtering strategy mattered more than index choice
- Documentation drives NL-to-SQL accuracy; four layers keep it safe
- Hybrid fusion did *not* help: reported as found

Limits and next steps

- 300 flights, one machine, one model, author-written questions
- Index-benchmark vectors are perturbed copies of real ones
- Event thresholds are heuristics; battery-low raises 23% false alarms on controls
- Next: 10,000+ logs, ArduPilot at scale, a concurrency benchmark